

LSTM

Long Short-Term Memory

The Complete Mathematical Reference

End-to-End Learning Materials

Forward Pass · Backpropagation · Gradients · Variants · Training · Colah's
Intuitions

Originally introduced by Hochreiter & Schmidhuber (1997)
Colah's visual intuitions (2015) incorporated in Sections 15–18

TABLE OF CONTENTS

1. Prerequisites & Notation
2. Activation Functions
3. Why LSTM? The Vanishing Gradient Problem
4. LSTM Architecture Overview
5. Forward Pass — All 6 Equations
6. Numerical Forward Pass Example
7. Loss Function — Softmax Cross-Entropy
8. Backpropagation Through Time (BPTT)
9. Gate Gradient Derivations
10. Weight Update Rules
11. Gradient Clipping
12. LSTM Variants
13. Training Best Practices
14. Quick Reference Card
15. The Conveyor Belt Intuition (Colah)
16. Coupled Gates & The Road to Attention
17. Gate-by-Gate Visual Walkthrough
18. Long-Term Dependency: A Full Example

SECTION 1

Prerequisites & Notation

What you need to know first

Symbol Reference Table

Symbol	Meaning	Dimension
$x(t)$	Input vector at time step t	$d \times 1$
$h(t)$	Hidden state (short-term memory) at time t	$n \times 1$
$C(t)$	Cell state (long-term memory) at time t	$n \times 1$
$f(t)$	Forget gate output	$n \times 1$
$i(t)$	Input gate output	$n \times 1$
$g(t)$	Candidate cell / Gate gate	$n \times 1$
$o(t)$	Output gate output	$n \times 1$
W_f	Weight matrix for forget gate	$n \times (n+d)$
W_i	Weight matrix for input gate	$n \times (n+d)$
W_g	Weight matrix for gate gate	$n \times (n+d)$
W_o	Weight matrix for output gate	$n \times (n+d)$
b_f, b_i, b_g, b_o	Bias vectors for each gate	$n \times 1$
$z(t)$	Concatenated input: $[h(t-1); x(t)]$	$(n+d) \times 1$
(x)	Element-wise (Hadamard) multiplication	—
σ	Sigmoid activation function	—
d	Input dimension	scalar
n	Hidden / cell state size (hyperparameter)	scalar
T	Sequence length (number of time steps)	scalar

Key Prerequisites

- Matrix multiplication and the dot product
- Basic calculus: derivatives and the chain rule
- Understanding of neural networks and gradient descent
- Familiarity with exponential functions (e^x)

SECTION 2

Activation Functions

The two building blocks of LSTM

Sigmoid $\sigma(z)$

The sigmoid function squishes any real number into the range **(0, 1)**. In LSTM, it acts as a **valve or gate** — 0 means block everything, 1 means let everything through.

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

z value	$\sigma(z)$ output	Interpretation
-10	≈ 0.000045	Gate nearly closed — forget/block
-2	≈ 0.119	Gate mostly closed
0	$= 0.500$	Gate half open
+2	≈ 0.881	Gate mostly open
+10	≈ 0.999955	Gate nearly fully open — pass through

KEY Sigmoid derivative: $\sigma'(z) = \sigma(z) * (1 - \sigma(z))$. This is used heavily in backpropagation. Maximum value of 0.25 at $z=0$.

Hyperbolic Tangent $\tanh(z)$

The tanh function squishes any real number into the range **(-1, +1)**. In LSTM, it acts as a **content creator** — positive values push memory one way, negative values the other.

$$\tanh(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)})$$

KEY Tanh derivative: $\tanh'(z) = 1 - \tanh^2(z)$. Also used heavily in backpropagation. Maximum value of 1.0 at $z=0$.

Derivatives Cheat Sheet — Memorise These

Function	Forward formula	Derivative (used in backprop)
$\sigma(z)$	$1 / (1 + e^{(-z)})$	$\sigma(z) * (1 - \sigma(z))$
$\tanh(z)$	$(e^z - e^{(-z)}) / (e^z + e^{(-z)})$	$1 - \tanh^2(z)$

SECTION 3

Why LSTM?

The Vanishing Gradient Problem

The Problem with Vanilla RNNs

In a standard RNN, the hidden state is updated as:

$$h(t) = \tanh(W * h(t-1) + U * x(t) + b)$$

During backpropagation, the gradient must travel back through every time step:

$$dL/dh(0) = dL/dh(T) * \prod_{t=1}^T dh(t)/dh(t-1)$$

Each factor contains $\tanh'(z)$, whose maximum value is 1.0 but typically much less. Over long sequences:

- $\tanh'(z)^{100} \rightarrow \approx 0$ (vanishing gradient: early steps get no signal)
- With large weights, gradients can also explode exponentially

CONSEQUENCE

Vanilla RNNs cannot learn long-range dependencies. The network forgets what happened more than ~10 time steps ago. LSTM was designed precisely to fix this.

How LSTM Solves This

LSTM introduces a separate Cell State $C(t)$ — a memory highway — that flows across time through addition, not multiplication:

$$C(t) = f(t) \odot C(t-1) + i(t) \odot g(t)$$

Gradients flow back through C via addition (+), not through repeated $\tanh$ multiplications. The forget gate $f(t)$ is learned — the network can set it near 1.0 to let gradients pass cleanly.

Architecture	Gradient path	Long-range memory?
Vanilla RNN	Through repeated $\tanh$ multiplications → vanishes	No (≈10 steps max)
LSTM	Through additive cell state + learned gates → stable	Yes (hundreds of steps)
GRU	Through additive update gate → stable	Yes (similar to LSTM)

SECTION 4

LSTM Architecture Overview

The big picture

Two Memory Streams

At every time step t , the LSTM maintains exactly **two vectors**:

Vector	Name	Role	Analogy
$C(t)$	Cell State	Long-term memory — stable, passes across many steps	Filing cabinet
$h(t)$	Hidden State	Short-term working memory — filtered version of $C(t)$	Sticky note

The Compact Matrix Formulation

In practice, $h(t-1)$ and $x(t)$ are concatenated into one vector $z(t)$, and all gate pre-activations are computed in a single matrix multiplication:

```
z(t) = [ h(t-1) ; x(t) ] (concatenate previous hidden state and
current input)
[ f ] [ sigma ] Forget gate
[ i ] = [ sigma ] * W^l * z(t) Input gate
[ g ] [ tanh ] Gate gate (candidate)
[ o ] [ sigma ] Output gate
```

W^l is a single large weight matrix covering all four gates for layer l .

Information Flow Summary

Gate	Symbol	Activation	Controls	Output range
Forget gate	$f(t)$	Sigmoid	How much of $C(t-1)$ to keep	0 to 1
Input gate	$i(t)$	Sigmoid	How much new info to write	0 to 1
Gate gate	$g(t)$	Tanh	What new content to potentially write	-1 to +1
Output gate	$o(t)$	Sigmoid	How much of $C(t)$ to expose as $h(t)$	0 to 1
Cell state	$C(t)$	None	Long-term memory store	Unbounded
Hidden state	$h(t)$	Tanh	Output and next-step input	-1 to +1

SECTION 5

Forward Pass

All 6 equations in order

Given: current input $x(t)$, previous hidden state $h(t-1)$, previous cell state $C(t-1)$, and learned weights W_f, W_i, W_g, W_o and biases b_f, b_i, b_g, b_o .

Step 1 — Forget Gate $f(t)$

PURPOSE Decide what percentage of the old cell state $C(t-1)$ to retain. Output near 0 = forget; near 1 = keep.

$$f(t) = \text{sigma}(W_f * [h(t-1); x(t)] + b_f)$$

Step 2 — Input Gate $i(t)$

PURPOSE Decide how much of the new candidate content $g(t)$ to actually write into the cell state.

$$i(t) = \text{sigma}(W_i * [h(t-1); x(t)] + b_i)$$

Step 3 — Gate Gate / Candidate $g(t)$

PURPOSE Create a draft of new content that could be added to memory. Uses tanh so values can be negative or positive (directional meaning).

$$g(t) = \text{tanh}(W_g * [h(t-1); x(t)] + b_g)$$

Step 4 — Update Cell State $C(t)$ — THE MEMORY HIGHWAY

PURPOSE Combine: erase part of old memory ($\times f$), write part of new content ($\times i$). Addition enables stable long-range gradients.

$$C(t) = f(t) (x) C(t-1) + i(t) (x) g(t)$$

Step 5 — Output Gate $o(t)$

PURPOSE Decide which parts of the cell state $C(t)$ to actually expose as the hidden state $h(t)$.

$$o(t) = \text{sigma}(W_o * [h(t-1); x(t)] + b_o)$$

Step 6 — Hidden State $h(t)$

PURPOSE Produce the actual output of this time step. tanh squishes $C(t)$ to $(-1,+1)$, then $o(t)$ filters how much is revealed.

$$h(t) = o(t) (x) \text{tanh}(C(t))$$

All 6 Equations — Complete Reference

$z(t) = [h(t-1); x(t)]$ concat inputs

$f(t) = \text{sigma}(W_f * z(t) + b_f)$ FORGET gate

$i(t) = \text{sigma}(W_i * z(t) + b_i)$ INPUT gate

$g(t) = \text{tanh}(W_g * z(t) + b_g)$ GATE gate

$C(t) = f(t)(x)C(t-1) + i(t)(x)g(t)$ CELL update

$o(t) = \text{sigma}(W_o * z(t) + b_o)$ OUTPUT gate

$h(t) = o(t)(x) \tanh(C(t))$ HIDDEN output

SECTION 6

Numerical Forward Pass Example

Concrete numbers through one time step

Setup

Setting	Value
Input dimension d	2 ($x = [0.5, -0.3]$)
Hidden size n	2
Previous hidden $h(t-1)$	$[0.1, -0.2]$
Previous cell $C(t-1)$	$[0.4, 0.1]$
Current input $x(t)$	$[0.5, -0.3]$
Concatenated $z(t)$	$[0.1, -0.2, 0.5, -0.3]$ (dim = 4)

Step 1 — Forget Gate

Assume $W_f * z(t) + b_f$ computes pre-activations: $pre_f = [1.2, -0.5]$

$$f(t) = \text{sigma}([1.2, -0.5])$$

$$f(t) = [1/(1+e^{(-1.2)}), 1/(1+e^{(0.5)})]$$

$$f(t) = [0.769, 0.378]$$

MEANING

Keep 76.9% of first cell dimension, only 37.8% of second. The LSTM is heavily discarding the second memory slot.

Step 2 — Input Gate

Assume $pre_i = [0.8, 1.5]$

$$i(t) = \text{sigma}([0.8, 1.5]) = [0.690, 0.818]$$

MEANING

Write 69% of new content into slot 1, and 81.8% into slot 2.

Step 3 — Gate Gate (Candidate)

Assume $pre_g = [0.3, -0.7]$

$$g(t) = \text{tanh}([0.3, -0.7]) = [0.291, -0.604]$$

MEANING

New content is slightly positive in slot 1, moderately negative in slot 2.

Step 4 — Update Cell State

$$c(t) = f(t)(x)C(t-1) + i(t)(x)g(t)$$

$$c(t) = [0.769, 0.378](x)[0.4, 0.1] + [0.690, 0.818](x)[0.291, -0.604]$$

$$C(t) = [0.308, 0.038] + [0.201, -0.494]$$

$$C(t) = [0.509, -0.456]$$

MEANING

Slot 1 increased slightly (0.4 to 0.509). Slot 2 flipped from slightly positive to moderately negative (0.1 to -0.456) due to the new content.

Step 5 — Output Gate

Assume $pre_o = [-0.4, 0.9]$

$$o(t) = \text{sigma}([-0.4, 0.9]) = [0.401, 0.711]$$

Step 6 — Hidden State

$$h(t) = o(t) (x) \tanh(C(t))$$

$$h(t) = [0.401, 0.711] (x) \tanh([0.509, -0.456])$$

$$h(t) = [0.401, 0.711] (x) [0.470, -0.426]$$

$$h(t) = [0.188, -0.303]$$

RESULT

The LSTM outputs $h(t) = [0.188, -0.303]$ and carries $C(t) = [0.509, -0.456]$ to the next time step.

SECTION 7

Loss Function

Softmax Cross-Entropy

From Hidden State to Prediction

After the final time step, the hidden state $h(T)$ is projected to vocabulary size V :

$$y = w_y * h(T) + b_y \text{ (un-normalized log-probabilities)}$$

Softmax Normalization

Convert raw scores y into a probability distribution:

$$p(k) = e^{(y_k)} / \sum_j e^{(y_j)} \text{ for each class } k \text{ in } \{1, \dots, V\}$$

Cross-Entropy Loss

$$L(t) = -\log(p(k))$$

$p(k)$ — probability given to correct class	$L(t)$ — loss	Interpretation
0.99	0.010	Very confident and correct — very low loss
0.50	0.693	Uncertain — moderate loss
0.10	2.303	Low probability to correct class — high loss
0.01	4.605	Totally wrong — very high loss

Total Loss Over a Sequence

$$L = \sum_{t=1 \text{ to } T} L(t) = -\sum_{t=1 \text{ to } T} \log(p_k(t))$$

Gradient of Softmax Loss

The gradient of L with respect to the raw output y is elegantly:

$$dL/dy_k = p(k) - 1_{\{k \text{ is correct class}\}}$$

Subtract 1 from the probability of the correct class; leave all others unchanged.

SECTION 8

Backpropagation Through Time

BPTT — How LSTM learns

The BPTT Concept

BPTT works by unrolling the LSTM across all T time steps, treating each step as a layer in a very deep network. Gradients are computed at the final step and propagated backwards through each gate and through time.

KEY INSIGHT

The cell state $C(t)$ is the gradient highway. Because $C(t) = f(x)C(t-1) + i(x)g$, the gradient flows back via addition — not through repeated $\tanh$ multiplications that shrink it.

Gradient w.r.t. Output Weight W_y

$$dL/dW_y = (dL/dy) * h(T)^T$$

Gradient w.r.t. Hidden State $h(t)$

The hidden state at each step receives gradient from TWO sources: from the output projection (final step), and from the next time step carrying future information backward.

$$dL/dh(T) = (dL/dy) * W_y^T + dh(next)$$

$$dL/dh(t) = dh(next) \text{ (for all other steps)}$$

Gradient w.r.t. Cell State $C(t)$ — The Critical Equation

$$dL/dC(t) = (dL/dh(t)) * o(t) * (1 - \tanh^2(C(t))) + dC(next)$$

Term	Meaning
$(dL/dh(t))$	Gradient arriving from the hidden state
$o(t)$	Output gate — scales how much $C(t)$ affects $h(t)$
$(1 - \tanh^2(C(t)))$	Derivative of $\tanh$ — the local gradient through $\tanh$
$dC(next)$	Gradient flowing back from the FUTURE cell state (key to LSTM!)

WHY THIS MATTERS

The term $dC(next)$ ADDS — not multiplies — the future gradient to the current cell gradient. This additive path is why LSTMs can propagate gradients across hundreds of time steps without vanishing.

SECTION 9

Gate Gradient Derivations

Step-by-step chain rule for every gate

Once we have $dL/dh(t)$ and $dL/dC(t)$, all gate gradients follow from the chain rule. Each gate has two gradient steps: through the gate output, then through its activation.

Output Gate Gradients

Step 1: Since $h(t) = o(t) \cdot \tanh(C(t))$:

$$dL/do(t) = (dL/dh(t)) \cdot \tanh(C(t))$$

Step 2: Gradient through sigmoid (o' is the pre-activation):

$$dL/do'(t) = (dL/do(t)) \cdot o(t) \cdot (1 - o(t)) \text{ [sigmoid derivative]}$$

Gate Gate (Candidate) Gradients

Step 1: Since $C(t) = f(t) \cdot C(t-1) + i(t) \cdot g(t)$:

$$dL/dg(t) = (dL/dC(t)) \cdot i(t)$$

Step 2: Gradient through tanh:

$$dL/dg'(t) = (dL/dg(t)) \cdot (1 - g(t)^2) \text{ [tanh derivative]}$$

Input Gate Gradients

Step 1: Since $C(t) = f(t) \cdot C(t-1) + i(t) \cdot g(t)$:

$$dL/di(t) = (dL/dC(t)) \cdot g(t)$$

Step 2: Gradient through sigmoid:

$$dL/di'(t) = (dL/di(t)) \cdot i(t) \cdot (1 - i(t)) \text{ [sigmoid derivative]}$$

Forget Gate Gradients

Step 1: Since $C(t) = f(t) \cdot C(t-1) + i(t) \cdot g(t)$:

$$dL/df(t) = (dL/dC(t)) \cdot C(t-1)$$

Step 2: Gradient through sigmoid:

$$dL/df'(t) = (dL/df(t)) \cdot f(t) \cdot (1 - f(t)) \text{ [sigmoid derivative]}$$

PATTERN

Every gate follows the same structure: (1) gradient = $dL/dC(t) \cdot (\text{other factor})$ and (2) multiply by the activation derivative (sigmoid' or tanh').

Gate	$dL/dgate$	$dL/dpre\text{-}activation$
$f(t)$ forget	$dL/dC(t) \cdot C(t-1)$	$(dL/df) \cdot f \cdot (1-f)$
$i(t)$ input	$dL/dC(t) \cdot g(t)$	$(dL/di) \cdot i \cdot (1-i)$
$g(t)$ gate	$dL/dC(t) \cdot i(t)$	$(dL/dg) \cdot (1-g^2)$

$o(t)$ output	$dL/dh(t) (x) \tanh(C(t))$	$(dL/do) (x) o (x) (1-o)$
---------------	----------------------------	---------------------------

SECTION 10

Weight Update Rules

How gradients become learning

Weight Gradients

Because all gates use $z(t) = [h(t-1); x(t)]$ as input, all weight gradients take the same form:

$$dL/dw_f = \text{SUM}(t) \ (dL/df'(t)) \ * \ z(t)^T$$

$$dL/dw_i = \text{SUM}(t) \ (dL/di'(t)) \ * \ z(t)^T$$

$$dL/dw_g = \text{SUM}(t) \ (dL/dg'(t)) \ * \ z(t)^T$$

$$dL/dw_o = \text{SUM}(t) \ (dL/do'(t)) \ * \ z(t)^T$$

The sum SUM accumulates gradients across all T time steps — this is the 'through time' part of BPTT.

Bias Gradients

$$dL/db_f = \text{SUM}(t) \ dL/df'(t)$$

$$dL/db_i = \text{SUM}(t) \ dL/di'(t)$$

$$dL/db_g = \text{SUM}(t) \ dL/dg'(t)$$

$$dL/db_o = \text{SUM}(t) \ dL/do'(t)$$

Gradient w.r.t. Input $z(t)$

$$dL/dz(t) = w_f^T \ * \ dL/df'(t) + w_i^T \ * \ dL/di'(t)$$

$$+ w_g^T \ * \ dL/dg'(t) + w_o^T \ * \ dL/do'(t)$$

SGD Weight Update

$$w_f \leftarrow w_f - \eta \ * \ dL/dw_f$$

Where η is the learning rate (typically 0.001 to 0.01). In practice, Adam optimizer is preferred:

Adam Optimizer Update

$$m_t = b1 \ * \ m_{(t-1)} + (1 - b1) \ * \ dL/dw \text{ (1st moment - mean)}$$

$$v_t = b2 \ * \ v_{(t-1)} + (1 - b2) \ * \ (dL/dw)^2 \text{ (2nd moment - variance)}$$

$$m_{\text{hat}} = m_t / (1 - b1^t) \text{ (bias correction)}$$

$$v_{\text{hat}} = v_t / (1 - b2^t)$$

$$w \leftarrow w - \eta \ * \ m_{\text{hat}} / (\text{sqrt}(v_{\text{hat}}) + \text{eps}) \text{ (weight update)}$$

Typical values: $b1=0.9$, $b2=0.999$, $\text{eps}=1e-8$, $\eta=0.001$.

SECTION 11

Gradient Clipping

Preventing exploding gradients

Even with LSTM's cell state highway, gradients can still explode — especially with deep networks. Gradient clipping is the standard fix.

Global Norm Clipping

$$\text{global_norm} = \sqrt{\sum_i (dL/d_{\theta_i})^2}$$

If $\text{global_norm} > \text{threshold}$, scale ALL gradients down proportionally:

$$G_{\text{clipped}} = G * (\text{max_norm} / \text{global_norm}) \text{ if } \text{global_norm} > \text{max_norm}$$

$$G_{\text{clipped}} = G \text{ otherwise}$$
KEY

Scaling preserves the direction of the gradient vector — only its magnitude is limited.

PROPERTY

This is why global norm clipping is preferred over per-parameter clipping.

PyTorch Implementation

```
# After loss.backward() – before optimizer.step()
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

max_norm setting	Effect
1.0	Standard for most LSTM tasks
5.0	More permissive — for tasks with naturally large gradients
0.1	Very aggressive — for very unstable training

SECTION 12

LSTM Variants

Evolution and alternatives

1. Peephole Connections (Gers & Schmidhuber, 2000)

Standard LSTM gates only see $h(t-1)$ and $x(t)$. Peephole connections give the gates direct access to cell state $C(t-1)$:

$f(t) = \text{sigma}(W_f*[h(t-1);x(t)] + W_{pf} (x) C(t-1) + b_f)$

$i(t) = \text{sigma}(W_i*[h(t-1);x(t)] + W_{pi} (x) C(t-1) + b_i)$

$o(t) = \text{sigma}(W_o*[h(t-1);x(t)] + W_{po} (x) C(t) + b_o)$

USE CASE Peepholes improve performance on tasks requiring precise timing or counting — speech recognition and music generation.

2. Gated Recurrent Unit — GRU (Cho et al., 2014)

GRU simplifies LSTM by merging cell state and hidden state, replacing forget+input gates with a single update gate:

$z(t) = \text{sigma}(W_z*[h(t-1);x(t)] + b_z)$ Update gate

$r(t) = \text{sigma}(W_r*[h(t-1);x(t)] + b_r)$ Reset gate

$h_tilde(t) = \text{tanh}(W_h*[r(t)(x)h(t-1);x(t)] + b_h)$ Candidate

$h(t) = (1 - z(t)) (x) h(t-1) + z(t) (x) h_tilde(t)$ Final hidden

Aspect	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Memory vectors	2 (C and h)	1 (h only)
Parameters	More (4 weight matrices per layer)	Fewer (3 weight matrices)
Speed	Slower	Faster (~25% fewer params)
Performance	Slightly better on complex tasks	Comparable on most tasks
Introduced	1997 (Hochreiter & Schmidhuber)	2014 (Cho et al.)

3. Bidirectional LSTM

Two LSTMs run simultaneously: one forward ($t=1$ to T) and one backward ($t=T$ to 1). Hidden states are concatenated:

$h(t) = [h_forward(t) ; h_backward(t)]$

USE CASE BiLSTM is ideal when the full sequence is available: sentiment analysis, named entity recognition, machine translation encoding.

4. Stacked (Deep) LSTM

$$\mathbf{x}^{(l+1)}(t) = \mathbf{h}^{(l)}(t) \text{ (output of layer } l \text{ feeds into layer } l+1)$$

Stacking typically uses 2–4 layers. Dropout is applied between layers to prevent overfitting.

SECTION 13

Training Best Practices

Making LSTM work in practice

Weight Initialization

Weight type	Recommended initialization	Reason
Input weights W_x	Glorot/Xavier uniform: $\pm\sqrt{6/(\text{fan_in}+\text{fan_out})}$	Balanced variance across layers
Recurrent weights W_h	Orthogonal initialization	Prevents gradient decay in hidden state
Forget gate bias b_f	Initialize to 1.0 (not 0)	Encourages remembering at start of training
Other biases	Initialize to 0	Standard practice

IMPORTANT Initializing the forget gate bias to 1.0 is a well-known trick that significantly speeds up LSTM learning. It biases the network to initially remember most of its memory, giving gradients a clear path backwards.

Regularization

- Dropout: apply between LSTM layers (not across time steps) — use dropout rate 0.2 to 0.5
- Recurrent dropout: drop connections within the LSTM across time — apply consistently
- L2 regularization: add $\lambda * ||W||^2$ to loss — typical $\lambda = 1e-4$ to $1e-5$

Hyperparameter Guidelines

Hyperparameter	Typical range	Notes
Hidden size n	64 — 1024	Larger = more capacity, more compute
Number of layers	1 — 4	2 layers usually optimal; use dropout between
Learning rate	$1e-3$ — $1e-4$	Adam optimizer; reduce on plateau
Gradient clip max_norm	1.0 — 5.0	1.0 is safe default
Sequence length T	Task dependent	Truncated BPTT: 35-100 steps typical
Batch size	16 — 128	Smaller = noisier but faster convergence
Dropout rate	0.2 — 0.5	Higher for smaller datasets

Truncated BPTT

- Forward pass over k steps, then backward pass over the same k steps
- Hidden state h and cell state C are carried forward across chunks
- Only gradients within each chunk are computed — beyond k steps are ignored
- Typical chunk size $k = 35$ to 100 steps

TRADEOFF

Truncated BPTT trades off some long-range gradient information for efficiency. In practice, models still learn long-range dependencies because the cell state carries information forward even without gradient flow.

SECTION 14

Quick Reference Card

Everything on one page

Forward Pass — 6 Equations

$$z(t) = [h(t-1); x(t)] \text{ concat inputs}$$

$$f(t) = \text{sigma}(W_f * z(t) + b_f) \text{ FORGET gate}$$

$$i(t) = \text{sigma}(W_i * z(t) + b_i) \text{ INPUT gate}$$

$$g(t) = \text{tanh}(W_g * z(t) + b_g) \text{ GATE gate}$$

$$C(t) = f(t)(x)C(t-1) + i(t)(x)g(t) \text{ CELL update}$$

$$o(t) = \text{sigma}(W_o * z(t) + b_o) \text{ OUTPUT gate}$$

$$h(t) = o(t) (x) \text{ tanh}(C(t)) \text{ HIDDEN output}$$

Backward Pass — Gate Gradients

$$dL/dC(t) = dL/dh(t) * o(t) * (1 - \text{tanh}^2(C(t))) + dC(\text{next}) \text{ CELL grad}$$

$$dL/do(t) = dL/dh(t) (x) \text{ tanh}(C(t)) \text{ output gate}$$

$$dL/dg(t) = dL/dC(t) (x) i(t) \text{ gate gate}$$

$$dL/di(t) = dL/dC(t) (x) g(t) \text{ input gate}$$

$$dL/df(t) = dL/dC(t) (x) C(t-1) \text{ forget gate}$$

$$\text{Apply sigmoid}' = \text{gate} * (1 - \text{gate}) \text{ or } \text{tanh}' = 1 - \text{gate}^2 \text{ activations}$$

Key Identities

Identity	Formula
Sigmoid derivative	$\text{sigma}'(z) = \text{sigma}(z) * (1 - \text{sigma}(z))$
Tanh derivative	$\text{tanh}'(z) = 1 - \text{tanh}^2(z)$
Softmax loss gradient	$dL/dy_k = p(k) - 1_{\{\text{correct class}\}}$
Gradient clipping	$G_{\text{clip}} = G * \min(1, \text{max_norm} / G)$

Architecture Lineage

RNN (1986) → **LSTM (1997)** → +forget gate (1999) → +peepholes (2000) → BiLSTM (2005) → **GRU (2014)** → **Transformer (2017)**

SECTION 15

The Conveyor Belt Intuition

Colah's landmark visual explanation

SOURCE This section draws on Christopher Olah's seminal blog post 'Understanding LSTM Networks' (2015), widely considered the clearest conceptual introduction to LSTMs ever written.

The Cell State as a Conveyor Belt

The most powerful insight in understanding LSTMs is to think of the cell state $C(t)$ not as a computation, but as a **conveyor belt** running through the entire sequence.

Imagine a factory assembly line. The conveyor belt moves product from one end to the other. Workers at each station can add or remove parts — but most of the time, the product travels through untouched. The cell state works exactly like this:

- It runs straight through the entire chain of time steps
- It has only minor, linear interactions at each step
- Information can flow along it largely unchanged across many steps
- Gates decide when and what to add or remove from the belt

KEY INSIGHT Because the cell state interacts linearly (via addition and element-wise multiplication by values close to 1), information does not get distorted as it travels. A tanh applied once is survivable — applied 100 times in sequence, it destroys signal. The conveyor belt avoids this.

What a Gate Actually Is

A gate in LSTM is simply: a sigmoid layer followed by element-wise multiplication.

```
gate_output = sigmoid(W * [h(t-1); x(t)] + b)
result = gate_output (x) information_to_control
```

Sigmoid output	Effect on information	Analogy
0.0	Completely blocked — nothing gets through	Gate fully closed
0.5	Half the signal passes through	Gate half open
1.0	Everything passes through unchanged	Gate fully open

The Language Model Pronoun Example — Gate by Gate

Colah uses a beautiful real-world example: a language model where the cell state stores the gender of the current subject to generate correct pronouns (he/she/they).

"The chef prepared the meal. He said it was..." → "The nurse arrived. She checked..."

Gate	What it does in this example	Mathematical action
FORGET gate $f(t)$	When 'nurse' is seen, forget gate erases stored gender of 'nurse' (masculine). Outputs near 0 for gender slot.	$f(t) \approx 0$ for gender slot $f(t)(x)C(t-1) \approx 0$ (old gender erased)
INPUT gate $i(t)$	Decides gender of 'nurse' IS important. Outputs near 1 for gender slot	$i(t) \approx 1$ for gender slot $i(t)(x)g(t) \approx g(t)$ (new gender written)
GATE gate $g(t)$	Creates vector encoding 'feminine gender'. Positive gate values for the gender slot.	$g(t) \approx +0.8$ for gender slot (feminine encoding)
CELL STATE $C(t)$	Old masculine erased, new feminine written. Rest of cell state unchanged on the conveyor belt.	$C(t) = f(t)C(t-1) + i(t)g(t)$ = new feminine gender (rest unchanged)
OUTPUT gate $o(t)$	When generating 'she', output gate opens for gender slot, exposing gender as hidden state.	$o(t) \approx 1$ for gender slot $h(t) = o(t)(x)\tanh(C(t))$ → exposes gender info

INSIGHT

Notice that the forget and input gates work together but independently. The network learns to set forget≈0 and input≈1 simultaneously when replacing information — but they are computed separately, giving the network flexibility to partially update.

SECTION 16

Coupled Gates & The Road to Attention

Variants and what came next

Coupled Forget-Input Gate

Instead of computing forget and input gates **independently**, we can **couple them** — making them complementary. The logic: we should only forget when we have something new to input, and only input when we choose to forget.

Standard LSTM (independent gates):

$$C(t) = f(t)(x)C(t-1) + i(t)(x)g(t)$$

$f(t)$ and $i(t)$ are computed separately — both can be 0.9, meaning 'keep 90% of old AND write 90% of new'.

Coupled LSTM (linked gates):

$$C(t) = f(t)(x)C(t-1) + (1 - f(t))(x)g(t)$$

Now there is only ONE gate to learn. When $f(t)=1$ (keep everything), automatically $(1-f(t))=0$ (write nothing). When $f(t)=0$ (forget everything), $(1-f(t))=1$ (write everything).

Variant	Parameters	Flexibility	When to use
Standard LSTM	4 weight matrices	Full: forget and input any combination	When partial updates are important (most tasks)
Coupled LSTM	3 weight matrices	Constrained: forget + input always sum to 1	When memory is zero-sum (replace, not accumulate)
GRU	3 weight matrices	Like coupled, but also merges cell+hidden	Speed-constrained tasks, smaller datasets

PRACTICAL NOTE Greff et al. (2015) benchmarked 8 LSTM variants on speech and music tasks and found none significantly outperformed standard LSTM. The coupled variant performed comparably while using fewer parameters.

The Unrolled RNN: Why Depth Matters

An RNN **unrolled through time** is equivalent to a very deep feedforward network — but with **shared weights at every layer**. A sequence of length T creates a network T layers deep. The LSTM's additive cell state is the bypass that makes this tractable.

Standard RNN unrolled (T=5):

$x(1) \rightarrow [\tanh] \rightarrow h(1) \rightarrow [\tanh] \rightarrow h(2) \rightarrow [\tanh] \rightarrow h(3) \rightarrow [\tanh] \rightarrow h(4) \rightarrow [\tanh] \rightarrow h(5) \rightarrow \text{output}$
Gradient: $x \tanh' x \tanh' x \tanh' x \tanh' x \tanh' \rightarrow \text{exponential decay!}$

LSTM unrolled (T=5):

$C(0) \text{ ----+---- } C(1) \text{ ----+---- } C(2) \text{ ----+---- } C(3) \text{ ----+---- } C(4) \text{ ----+---- } C(5)$
 $\wedge f,i,g \wedge f,i,g \wedge f,i,g \wedge f,i,g \wedge f,i,g$
Gradient through C: just addition! Flows back cleanly.

From LSTM to Attention — What Came Next

Colah's blog ends with a prescient note: the next big step after LSTM is **attention**. Understanding why helps solidify understanding of LSTM's own limitations.

Mechanism	How it accesses information	Limitation
Vanilla RNN	Only the most recent hidden state $h(t)$	Forgets early information entirely
LSTM	Cell state $C(t)$ — compressed summary of all prior steps	Summary is fixed-size; distant details can fade
Attention	Directly queries ALL prior hidden states with learned weights	No compression — can focus on any past step

HISTORICAL ARC

LSTM (1997) → Attention (2015) → Transformer (2017). Each step gave the network more direct access to past information: LSTM compresses it into a highway, attention indexes it directly, transformers attend to everything simultaneously.

SECTION 17

Gate-by-Gate Visual Walkthrough

The LSTM diagram explained step by step

Colah's blog is famous for its diagrams. This section describes what each diagram shows, making the visual intuition explicit in text and math.

Diagram 1: The Repeating Module — RNN vs LSTM

Architecture	Inside the repeating module	Outputs per
Standard RNN	Single tanh layer: $h(t) = \tanh(W*[h(t-1);x(t)] + b)$	$h(t)$ only
LSTM	Four interacting layers: f, i, g, o gates + cell state update	$h(t)$ and $C(t)$

Diagram 2: The Cell State Line (The Conveyor Belt)

Cell state — the horizontal highway:

$C(t-1)$  $\rightarrow C(t)$
^ only touched by $f(t)$ and $i(t)(x)g(t)$

This horizontal line is the most important element. It runs **above** the gate computations, touching them only through one multiplication ($\times f$) and one addition ($+ i(x)g$). This is why gradients can flow back so cleanly.

Diagram 3: The Forget Gate

Forget gate wiring:

$h(t-1)$ --+
+---> [sigma] ---> $f(t)$ ---> (x) ---> $C(t-1)$ scaled
 $x(t)$ ---+

The (x) on the cell state line is the **only way the forget gate touches memory**. $f(t)=0.0$ at position j means $C(t-1)[j]$ is completely erased. $f(t)=1.0$ means it passes through unchanged.

Diagram 4: The Input Gate + Gate Gate

Input + Gate gate wiring:

$h(t-1)$ --+---> [sigma] ---> $i(t)$ --+
| +---> (x) --> new content
 $x(t)$ ---+---> [tanh] ---> $g(t)$ --+

Two separate paths from the same input $z(t)$: one through **sigmoid** (how much to write) and one through **tanh** (what to write). Their product $i(t)(x)g(t)$ gets added to the cell state.

Diagram 5: Cell State Update

Cell state update:

from forget: $f(t) (x) C(t-1)$ --+
+---> [+] --> $C(t)$ (NEW cell state)
from input: $i(t) (x) g(t)$ --+

Diagram 6: Output Gate

```
h(t-1) --+
+--> [sigma] --> o(t) --> (x) --> h(t)
x(t) ----+ C(t) --> [tanh] --+
```

Complete LSTM Cell — Full Assembled Diagram

[illegible]

SECTION 18

Long-Term Dependency: A Full Example

From first word to final prediction

Colah illustrates LSTM's power with a concrete example of long-term dependency. Let us trace through the full mathematical picture.

The Core Challenge

"I grew up in France ... I speak fluent ____"

Information needed	Where it appears	Distance
Topic: language	'speak fluent' (recent context)	Short-range — easy for any RNN
Which language	'France' (early in sentence)	Long-range — hard for vanilla RNN

What Happens Without LSTM (Vanilla RNN)

$$h(T) = \tanh(W * h(T-1) + U * x(T) + b)$$

$$h(T-1) = \tanh(W * h(T-2) + \dots) \leftarrow \text{depends on } h(T-2) \dots$$

$$h(T-k) \approx W^k * h(0) + \dots \leftarrow \text{exponentially decaying!}$$

The information about 'France', encoded in $h(\text{early})$, has been multiplied by W and passed through $\tanh$ many times. Its signal is essentially zero.

What Happens With LSTM

When the LSTM processes 'France' early in the sequence:

- Gate $g(t)$ encodes 'France' into a vector representing 'French-speaking country'
- Input gate $i(t)$ decides this is important — outputs values near 1
- This information is written into cell state: $C(t) += i(t)(x)g(t)$

As the sentence continues with irrelevant words:

- The forget gate keeps $f(t) \approx 1$ for the 'France' slot
- Cell state carries 'France-country' forward: $C(t) = f(t)(x)C(t-1) \approx C(t-1)$
- Gradient: $dC(t)/dC(t\text{-early}) \approx \text{prod}(f(k)) \approx 1^k = 1$ (no decay!)

When the model reaches 'speak fluent ____':

- Output gate opens for stored country information
- $h(T) = o(T)(x)\tanh(C(T))$ exposes 'French-speaking country' to prediction layer
- Softmax correctly gives high probability to 'French'

THE MATH BEHIND THE MAGIC

$dL/dC(\text{early}) = dL/dC(T) * \text{prod}(t=\text{early}+1 \text{ to } T) f(t)$. If forget gates learned to stay near 1.0 for this memory slot, the product $\approx 1^{(T-\text{early})} = 1$. The gradient reaches all the way back to 'France' without vanishing!

The Gap Problem Quantified

Gap (k steps)	Vanilla RNN gradient	LSTM gradient ($f \approx 0.9$)	LSTM gradient ($f \approx 1.0$)
1 step	$\tanh'(z)^1 \approx 0.42$	$0.9^1 = 0.90$	$1.0^1 = 1.00$
5 steps	$\tanh'(z)^5 \approx 0.013$	$0.9^5 = 0.59$	$1.0^5 = 1.00$
10 steps	$\tanh'(z)^{10} \approx 0.0002$	$0.9^{10} = 0.35$	$1.0^{10} = 1.00$
50 steps	$\tanh'(z)^{50} \approx 0$	$0.9^{50} = 0.005$	$1.0^{50} = 1.00$
100 steps	$\tanh'(z)^{100} \approx 0$	$0.9^{100} \approx 0.000$	$1.0^{100} = 1.00$

When the forget gate learns to set $f \approx 1.0$ for an important memory slot, the gradient flows back **perfectly regardless of sequence length**. This is the mathematical proof that LSTM solves the long-term dependency problem.

Summary: The Three Things LSTM Gets Right

Problem	LSTM Solution	Mechanism
Vanishing gradient	Cell state gradient highway	Additive update: $dC(t)/dC(t-1) = f(t) \approx 1$
Forgetting old info	Selective memory via forget gate	$f(t)$ near 1 preserves; $f(t)$ near 0 erases
Writing relevant new info	Selective writing via input + gate	$g(t) \cdot g(t)$ adds only chosen content to $C(t)$

FINAL INSIGHT

LSTM does not remember everything — that would be noise. It learns WHAT to remember, WHEN to remember it, and WHEN to forget it. The mathematics of the three gates are precisely designed to make these decisions differentiable, so they can be learned from data via gradient descent.